

ENTENDENDO A LÓGICA DE PROGRAMAÇÃO NO PROJETO DE UMA INJEÇÃO
ELETRÔNICA QUE REALIZA CÁLCULO DO TEMPO DE INJEÇÃO A PARTIR DOS
PARÂMETROS DE PULSOS E TEMPERATURA

FACULDADE DE TECNOLOGIA DE SOROCABA

DISCIPLINA: TÓPICOS AVANÇADOS PARA MICROCONTROLADORES

PROJETO ELABORADO POR: PEDRO HENRIQUE VELLOSO DA SILVA

SIMULAÇÃO: PROTEUS 8.13

SUMÁRIO

1.COMONENTES

1.1 Microcontrolador Pic18f4520

1.2 Potênciometro

1.3 Dclock (Proteus)

1.4 Leds

1.5 LCD

2. PROGRAMAÇÃO

2.1 Programando potenciômetro

2.2 Programando Dclock

2.3 Programando os leds

2.4 Programando LCD

2.5 Programação final

3. PROJETO FINALIZADO

3.1 Montagem do circuito no Proteus

3.2 Programação completa

1 COMPONENTES

1.1 Microcontroladores PIC18F4520

O PIC18F4520 é o microcontrolador responsável pelo funcionamento principal do projeto. Ele realiza a leitura dos sensores simulados, faz os cálculos de RPM e temperatura, controla os LEDs e envia informações ao display LCD.

O microcontrolador possui:

Conversor ADC interno;

Portas digitais de entrada e saída;

Comunicação serial;

Temporizadores;

Memória Flash.

No projeto ele foi utilizado para:

Ler o potenciômetro no pino RA0;

Receber pulsos do Dclock no RB0;

Acionar os LEDs dos bicos injetores;

1.2 Potenciômetro

O potenciômetro foi utilizado para simular o sensor de temperatura do motor.

Ele fornece uma tensão variável de 0V até 5V para o pino AN0 do PIC.

O ADC converte essa tensão em um valor digital entre:

0 até 1023.

Depois o programa transforma esse valor em uma temperatura entre:

20°C até 110°C

1.3 Dclock (Proteus)

O componente Dclock do Proteus foi utilizado para simular o sensor de rotação do motor.

Ele envia pulsos digitais constantes para o pino RB0 do microcontrolador.

A frequência do Dclock determina o RPM calculado pelo sistema.

Exemplo:

10 Hz → 600 RPM

20 Hz → 1200 RPM

50 Hz → 3000 RPM

1.4 LEDs

Os LEDs foram utilizados para simular os bicos injetores.

Foram conectados aos pinos:

RD0

RD1

RD2

RD3

O tempo em que os LEDs permanecem ligados representa o tempo de injeção calculado pelo sistema.

1.5 LCD

O LCD 16x2 foi utilizado para exibir as informações do sistema em tempo real.

No display são mostrados:

RPM do motor;

Temperatura simulada.

O LCD foi configurado em modo 4 bits para reduzir a quantidade de fios utilizados no circuito.

Pinos utilizados:

LCD PIC18F4520

RS RB1

EN RB2

D4 RD4

D5 RD5

D6 RD6

D7 RD7

2. PROGRAMAÇÃO

2.1 Programando potenciômetro

O potenciômetro foi configurado utilizando o módulo ADC do PIC18F4520.

O ADC transforma um sinal analógico em valor digital.

Configuração ADC

```
ADCON1 = 0b00001110;  
ADCON2 = 0b10101001;  
ADCON0 = 0b00000001;
```

Explicação

ADCON1 define quais pinos serão analógicos;

ADCON2 configura velocidade e alinhamento do ADC;

ADCON0 liga o ADC e seleciona o canal AN0.

Leitura ADC

```
unsigned int Ler_ADC()  
{  
    GO_DONE = 1;  
  
    while(GO_DONE);  
  
    return ((ADRESH << 8) + ADRESL);  
}
```

A função inicia a conversão analógica e aguarda o término da leitura.

Conversão da temperatura

```
temperatura = 20 + ((unsigned long)valor_adc * 90 / 1023);
```

Essa fórmula converte o valor ADC em temperatura.

ADC	Temperatura
0	20°C
512	65°C
1023	110°C

2.2 Programando Dclock

O Dclock envia pulsos digitais ao pino RB0.

O programa detecta bordas de subida.

Contagem de pulsos

```
if(PORTBbits.RB0 == 1 &&  
    pulso_anterior == 0)  
{  
    pulsos++;  
}
```

Quando ocorre mudança de 0 para 1:

um pulso é contado.

Cálculo do RPM

```
rpm = pulsos * 600;
```

O sistema conta pulsos durante 100 ms.

Por isso multiplica:

por 10 para converter para 1 segundo;

por 60 para converter Hz em RPM.

2.3 Programando os LEDs

Os LEDs simulam os bicos injetores.

Tempo base

```
tempo_base = 120 - temperatura;
```

Quanto menor a temperatura:

maior o tempo de injeção.

Correção RPM

```
tempo_injecao = tempo_base + (rpm / 100);
```

Quanto maior o RPM:

maior o tempo de acionamento.

Acionamento dos LEDs

```
LATDbits.LATD0 = 1;  
LATDbits.LATD1 = 1;  
LATDbits.LATD2 = 1;  
LATDbits.LATD3 = 1;
```

Desligamento

```
LATDbits.LATD0 = 0;  
LATDbits.LATD1 = 0;  
LATDbits.LATD2 = 0;  
LATDbits.LATD3 = 0;
```

2.4 Programando LCD

O LCD foi configurado em modo 4 bits.

Definição dos pinos

```
#define RS LATBbits.LATB1  
#define EN LATBbits.LATB2  
#define D4 LATDbits.LATD4  
#define D5 LATDbits.LATD5  
#define D6 LATDbits.LATD6  
#define D7 LATDbits.LATD7
```

Função do RS

O pino RS define:

RS = 0 → comando;

RS = 1 → caractere.

Função EN

O EN funciona como um pulso de confirmação.

Quando o EN muda de 1 para 0:

o LCD lê os dados.

Exibição do RPM

```
sprintf(texto1, "RPM:%4u ", rpm);
```

Exibição da temperatura

```
sprintf(texto2, "TEMP:%3uC ", temperatura);
```

A biblioteca stdio.h foi utilizada para permitir o uso da função sprintf().

2.5 Programação final

Na programação final:

O PIC lê o potenciômetro;

Calcula a temperatura;

Conta pulsos do Dclock;

Calcula o RPM;

Calcula o tempo de injeção;

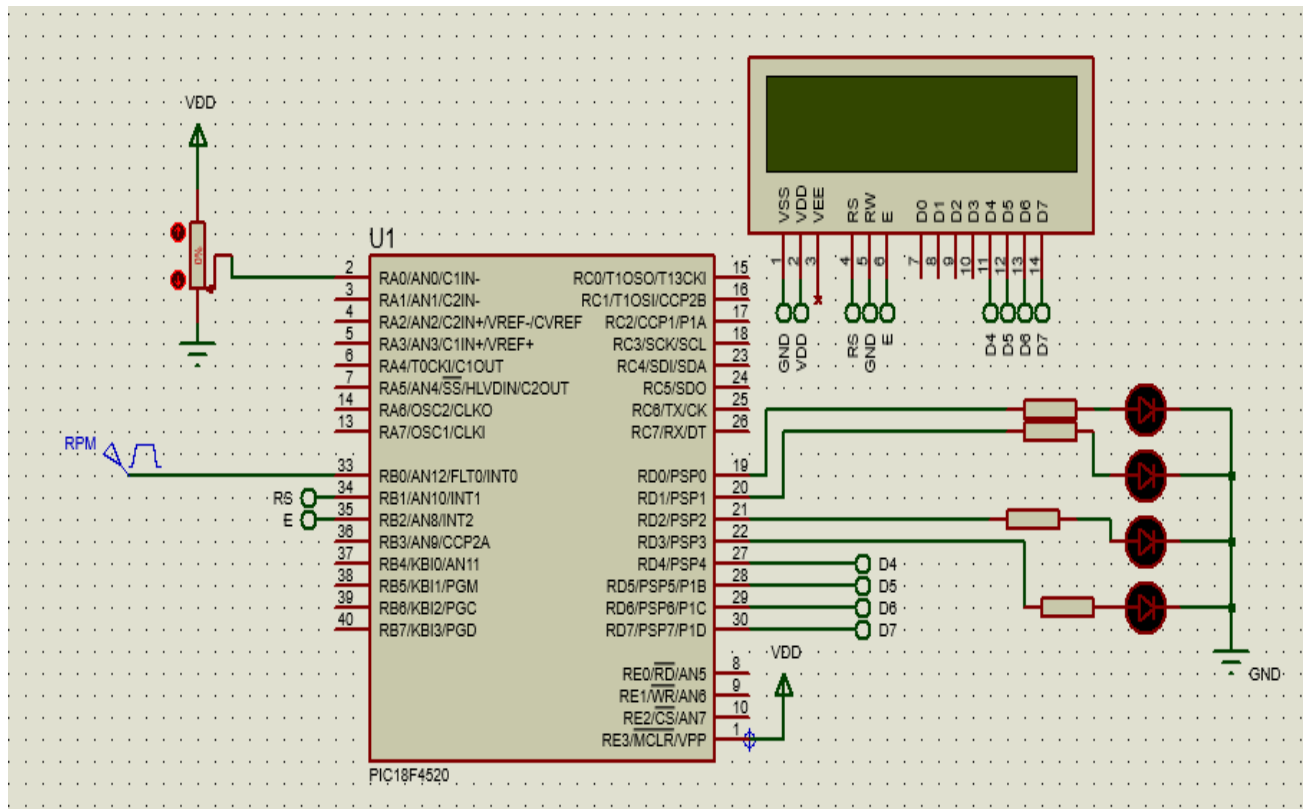
Aciona os LEDs;

Atualiza o LCD.

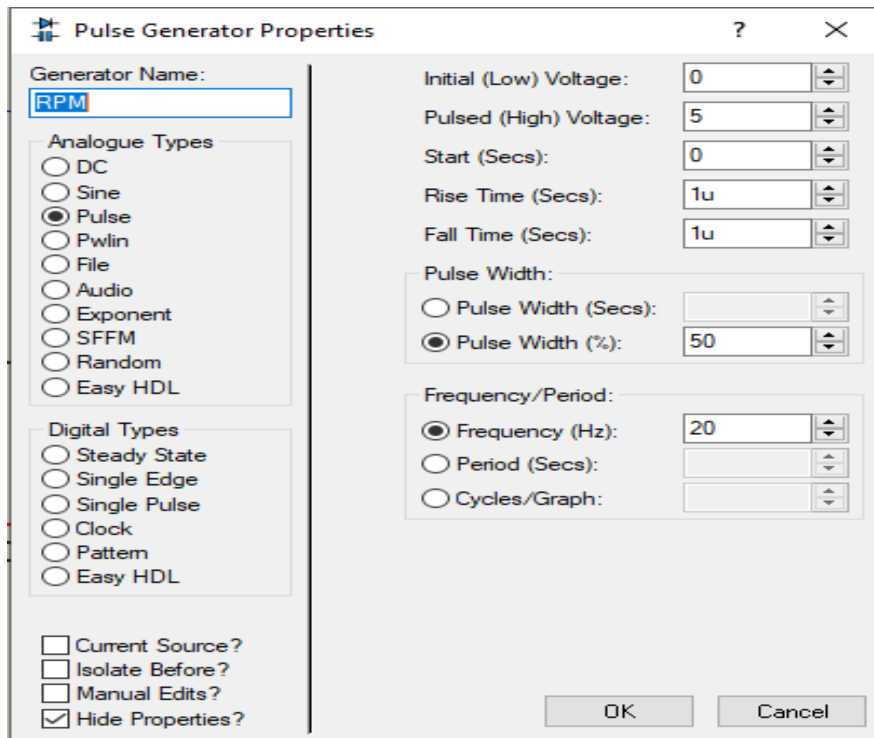
Todo o processo ocorre continuamente dentro do while(1).

3 PROJETO FINALIZADO

3.1 Montagem do circuito



Montagem do circuito. Fonte: Do autor.



Configuração do Dclock (Proteus). Fonte: Do autor.

3.2 Programação completa

```
#include <pic18f4520.h> //o pic escolhido para este programa
```

```
#include <xc.h> //inclue a biblioteca XC8
```

```
#include <stdio.h> //inclui entrada e saída padrão
```

```
#define _XTAL_FREQ 8000000 //define frequência para 8 MHz
```

```
#define RS LATBbits.LATB1 //pino LCD
```

```
#define EN LATBbits.LATB2 //pino LCD
```

```
#define D4 LATDbits.LATD4 //pino LCD
```

```
#define D5 LATDbits.LATD5 //pino LCD
```

```
#define D6 LATDbits.LATD6 //pino LCD
```

```
#define D7 LATDbits.LATD7 //pino LCD
```

```
#pragma config WDT = OFF //Desativa Watch Dog
```

```
#pragma config PBDEN = OFF //Inicia PORTB como digital
```

```

unsigned int valor_adc; //guarda valor lido no adc
unsigned int temperatura; //guarda valor da temperatura calculada
unsigned char pulso_anterior = 0; //guarda estado anterior do RBO
unsigned long pulsos = 0; //conta pulsos recebidos
unsigned int rpm = 0; //cria variável de rpm
unsigned int tempo_base; //cria variável de tempo base para o cálculo da injeção
unsigned int tempo_injecao; //o tempo de injeção que será executado
char texto1[16]; //texto para lcd linha 1, 16 caracteres
char texto2[16]; //texto para lcd linha 2, 16 caracteres

void ADC_init()
{

    ADCON1 = 0b00001110; //configura ADCON1 e deixa AN0 como analógico e o restante
    digital
    ADCON2 = 0b10101001;
    // ADFM = 1 -> resultado justificado a direita
    // ACQT = 010 -> tempo de aquisicao
    // ADCS = 001 -> clock do ADC
    ADCON0 = 0b00000001;
    // Canal AN0 selecionado
    // ADC ligado
}

unsigned int Ler_ADC()
{
    GO_DONE = 1; //inicia conversão ADC
    while(GO_DONE); //espera conversão terminar
    return ((ADRESH << 8) + ADRESL); //junta ADRESH e ADRESL
}

```

```
}
```

```
void LCD_Nibble(unsigned char nibble)
```

```
{
```

```
    D4 = (nibble >> 0) & 1;
```

```
    D5 = (nibble >> 1) & 1;
```

```
    D6 = (nibble >> 2) & 1;
```

```
    D7 = (nibble >> 3) & 1;
```

```
    EN = 1;
```

```
    __delay_us(50);
```

```
    EN = 0;
```

```
}
```

```
void LCD_Comando(unsigned char cmd)
```

```
{
```

```
    RS = 0;
```

```
    LCD_Nibble(cmd >> 4);
```

```
    LCD_Nibble(cmd & 0x0F);
```

```
    __delay_ms(2);
```

```
}
```

```
void LCD_Char(unsigned char data)
```

```
{
```

```
    RS = 1;
```

```
    LCD_Nibble(data >> 4);
```

```
    LCD_Nibble(data & 0x0F);
```

```
    __delay_us(20);
```

```
}
```

```
void LCD_String(const char *texto)
```

```
{
```

```
    while(*texto)
```

```
    {
```

```
        LCD_Char(*texto);
```

```
        texto++;
```

```
    }
```

```
}
```

```
void LCD_Init()
```

```
{
```

```
    TRISB1 = 0;
```

```
    TRISB2 = 0;
```

```
    TRISD4 = 0;
```

```
    TRISD5 = 0;
```

```

TRISD6 = 0;
TRISD7 = 0;

__delay_ms(1);

LCD_Nibble(0x03);

LCD_Nibble(0x03);

LCD_Nibble(0x03);

LCD_Nibble(0x02);

LCD_Comando(0x28);

LCD_Comando(0x0C);

LCD_Comando(0x06);

//LCD_Comando(0x01);

__delay_ms(1);
}
void LCD_Set_Cursor(unsigned char linha,
                    unsigned char coluna)
{
    unsigned char posicao;

    if(linha == 1)

```

```

{
    posicao = 0x80 + coluna - 1;
}
else
{
    posicao = 0xC0 + coluna - 1;
}

LCD_Comando(posicao);
}

```

```

void main (void)
{
    TRISD = 0b00000000; //configura todas as portas em PORTD como saídas
    TRISB0 = 1; //Deixa RB0 como entrada

    ADC_init();          //inicializa ADC
    LCD_Init();

    while(1)
    {

        valor_adc = Ler_ADC(); // Le potenciometro
        // Formula da temperatura:
        // 20 ate 110 graus
    }
}

```

```
temperatura = 20 + ((unsigned long)valor_adc * 90 / 1023);
```

```
/* EXEMPLOS:
```

```
ADC = 0
```

```
temperatura = 20°C
```

```
ADC = 512
```

```
temperatura ~ 65°C
```

```
ADC = 1023
```

```
temperatura = 110°C
```

```
*/
```

```
//-----Leitura do DCLOCK-----
```

```
// Detecta borda de subida
```

```
    pulsos = 0;
```

```
// Conta pulsos durante 1 segundo
```

```
for(int i = 0; i < 100; i++)
```

```
    {
```

```
// Detecta borda de subida
```

```
if(PORTBbits.RB0 == 1 &&
```

```
pulso_anterior == 0)
```

```
    {
```

```
pulsos++;
```

```
    }
```

```
pulso_anterior = PORTBbits.RB0;
```

```
__delay_ms(1);
```

```
    }
```

```
rpm = pulsos * 600;
```



```
//===== TEMPO BASE =====
```

```
// Quanto mais frio, maior o tempo
```

```
tempo_base = 120 - temperatura;
```

```
//===== CORRECAO RPM =====
```

```
tempo_injecao = tempo_base + (rpm / 100);
```

```
// Liga os 4 bicos
```

```
LATDbits.LATD0 = 1;
```

```
LATDbits.LATD1 = 1;
```

```
LATDbits.LATD2 = 1;
```

```
LATDbits.LATD3 = 1;
```

```
// Mantem ligados pelo tempo calculado
```

```
for(int i = 0; i < tempo_injecao; i++)
```

```
{
```

```
    __delay_ms(1);
```

```
}
```

```
// Desliga os bicos
```

```
LATDbits.LATD0 = 0;
```

```
LATDbits.LATD1 = 0;
```

```
LATDbits.LATD2 = 0;
```

```
LATDbits.LATD3 = 0;
```

```
__delay_ms(1);      // Pequeno atraso
```

```
LCD_Set_Cursor(1,1);
```

```
sprintf(texto1, "RPM:%4u ", rpm);
```

```
LCD_Set_Cursor(1,1);
```

```
LCD_String(texto1);
```

```
sprintf(texto2, "TEMP:%3uC ", temperatura);
```

```
LCD_Set_Cursor(2,1);
```

```
LCD_String(texto2);
```

```
}
```

```
}
```